📚 Verilog Basics Overview

Key Points

- Understanding lexical tokens and identifiers.
- Writing and instantiating modules with ports.
- Using value sets, operators, and modeling styles.

📘 Verilog Overview

> **Verilog HDL** is one of the two most common hardware description languages (HDL) used by integrated-circuit designers (the other is VHDL).
> Designs written in HDL are **technology-independent**, easier to debug, and more readable than schematic diagrams, especially for large circuits.

🔤 Lexical Tokens

␣ White Space

- Spaces, tabs, new-lines, and form-feeds separate tokens.
- Statements may span multiple lines without special continuation characters.

💬 Comments

- **Line comment:** // … end of line.
- **Block comment:** /* … */ (can span multiple lines).

🏷️ Identifiers

- Begin with a **letter** or **underscore** (not a digit or $).
- May contain letters, digits, underscores; case-sensitive.
- **Valid examples:** comparator_input, Compout$p, _datax64
- **Invalid examples:** &comparator1, #compoutp, 45data

🔑 Keywords

- Reserved words with special meaning (e.g., assign, case, while, wire, reg, module).
- Must **not** be used as identifiers.

🧩 Case Sensitivity

- All keywords are **lower-case**.
- Identifiers that start with an upper-case letter are distinct from their lower-case versions.

> Example: wire (keyword) vs. WIRE (identifier).

📦 Modules

⬛ Module Declaration

> A **module** is the primary design entity. Syntax:

```
module  (port_list);
    // port declarations
    // internal statements
endmodule
```

- Begins with module, ends with endmodule.
- Ports are listed inside parentheses and terminated with ;.

🔌 Ports

- Types: **input**, **output**, **inout**.
- Syntax example:

```
module comparator (input x, input y, output z);
    assign z = (~x & ~y) | (x & y);
endmodule
```

- Alternate style (separate port declarations) is also legal.
- **Default width** = 1 bit; specify width with [msb:lsb].

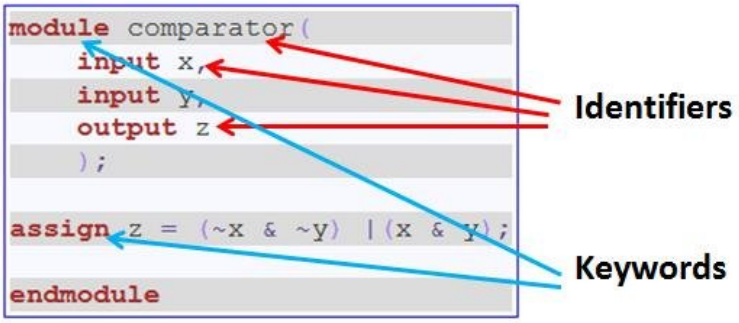
Port-type rules

Port	Internal type	External connection
input	must be a **net**	may connect to reg or net

output	may be **net** or **reg**	must connect to a **net**
inout	always a **net**	always a **net**

📸 Visual Example – Comparator Module

The diagram below highlights identifiers (x, y, z, comparator) and keywords (module, input, output, assign, endmodule).



The arrows point to each part of the code, reinforcing how the module's ports and assignment are structured.

⚙️ Value Set

Verilog uses four fundamental logic values:

Symbol	Meaning
0	Logic low / false
1	Logic high / true
x	Unknown
z	High-impedance (tri-state)

x and z have limited synthesis use.

🔢 Number Representation

- **Syntax:** '
  - size – decimal bit width (defaults to ≥ 32).

- base – d/D (decimal, default), h/H (hex), o/O (octal), b/B (binary).
- number – the value in the chosen base.

Example	Decimal	Binary
4'd3	3	0011
3'hA	10	1010
8'o26	22	00010110
5'b111	7	00111
8'bx1101	–	xxxx1101

Numbers whose most-significant bit is x or z are sign-extended with that value.

🔧 Built-in Primitives

Logic Gates (single output, one-or-more inputs)

Primitive	Truth table (A,B → Y)
and	00→0, 01→0, 10→0, 11→1
or	00→0, 01→1, 10→1, 11→1
xor	00→0, 01→1, 10→1, 11→0
nand	00→1, 01→1, 10→1, 11→0
nor	00→1, 01→0, 10→0, 11→0
xnor	00→1, 01→0, 10→0, 11→1

- The **first port** in the list is the **output**.

Buffer & Inverter

Primitive	I/O	Behaviour
buf	input → output	Passes signal unchanged
not	input → output	Logical inversion

Conditional Primitives (bufif, notif)

Primitive	Ports (output, data, control)	Meaning
bufif0	output, data, control	Drives data when control = 0, otherwise high-Z
bufif1	output, data, control	Drives data when control = 1, otherwise high-Z
notif0	output, data, control	Drives inverted data when control = 0
notif1	output, data, control	Drives inverted data when control = 1

- **L** = 0 or z, **H** = 1 or z; transitions to L/H become **x**.

📊 Data Types

🌐 Net Types (model physical connections)

- Declared with wire (default for ports) or tri.
- Do **not store** values; they reflect the value driven by sources.
- Unconnected nets resolve to z.

Net Type	Typical Use	Driver rule
wire / tri	Single driver (wire) or multiple drivers (tri)	Continuous driving required
wand / triand	Wired-AND; result is 0 if **any** driver is 0	No strength declaration needed
wor / trior	Wired-OR; result is 1 if **any** driver is 1	–
tri0	Pull-down; defaults to 0 when undriven	–
tri1	Pull-up; defaults to 1 when undriven	–
supply0 / supply1	Power supplies with fixed strength	–
trireg	Models capacitive net; retains previous value on z	–

	driver	

📩 Register Types (store values)

Type	Width	Signed?	Initialization	Assignment
reg	Specified range or 1 bit	Unsigned (unless declared signed)	x (unknown)	Procedural (always, initial)
integer	32 bit	**Signed**	x	Procedural
time	64 bit	Unsigned	x	Procedural
real / realtime	64-bit floating	–	**0**	Procedural (limited operator support)

Important notes

- Registers can only be updated by **procedural statements**.
- real/realtime cannot be used with many bitwise or shift operators.
- reg, integer, and time start as x; real/realtime start as 0.

Example Declarations

```
reg scal;                // 1-bit register
reg [7:0] vect;          // 8-bit vector register
reg [7:0] mem [31:0];    // 32-word memory, each 8 bits
integer i;
integer i_mem [7:0];
time t;
time t_mem [3:0];
real r;
realtime rt1, rt2;
```

📏 Vectors

- Declared with [msb:lsb] (most-significant to least-significant).
- Order may be **high-to-low** or **low-to-high**; the left number is always the **msb**.
- Only **net** and **reg** types may be vectors; illegal for integer, real, realtime, time.

Part-Selects

```
wire [31:0] busA, busB;
busA[7]          // single bit
busA[2:0]        // three LSBs (bits 2,1,0)
```

Variable Part-Selects (+: and -:)

- Syntax: [+ :] (increment) or [- :] (decrement).
- width must be a constant; start may be variable.

```
reg [255:0] data1;
reg [0:255] data2;
reg [7:0]   byte;

byte = data1[31-:8];   // bits 31 downto 24
byte = data1[24+:8];   // bits 24 upto 31
byte = data2[31-:8];   // bits 24 upto 31 (because data2 is low-to-high)
byte = data2[24+:8];   // bits 24 upto 31
```

Key points

- Both msb and lsb expressions must be constant.
- Vectors only for wire/reg.

⚙️ Operators

📃 Operator Categories

- **Arithmetic:** + - * / %
- **Relational:** < <= > >=
- **Logical:** ! && ||
- **Equality:** == != === !==
- **Bit-wise:** ~ & | ^ ~^ ^~
- **Reduction:** & ~& | ~| ^ ~^ ^~ (single-bit result)
- **Shift:** << >>
- **Conditional:** ?:
- **Concatenation:** { } , {{ }}
- **Replication:** {N{expr}}

⏫ Operator Precedence (high → low)

1. Unary (+ - ! ~)
2. Arithmetic (* / %)
3. Additive (+ -)
4. Shift (<< >>)
5. Relational (< <= > >=)
6. Equality (== != === !==)
7. Bit-wise AND/OR/XOR (& ^ | ~& ~| ^~ ~^)
8. Logical AND (&&)
9. Logical OR (||)
10. Conditional (?:)

🧮 Example of Arithmetic & Relational Use

```
module operators;
    reg [3:0] a, b;
    initial begin
        a = 3; b = 5;
        $display(a + b);   // 8
        $display(a * b);   // 15
        $display(a / b);   // 0
        $display(b / a);   // 1
        $display(a % b);   // 3
        $display(7 % 3);   // 1
        $display(7 % -3);  // 1
        $display(-7 % 3);  // -1
        $display(-7 % -3); // -1
    end
endmodule
```

🔀 Reduction Operators Table

Operand	&	~&	|	~|	^	^~/~^
4'b0000	0	1	0	1	0	1
4'b0001	0	1	1	0	1	0
4'b0011	0	1	1	0	0	1
4'b0111	0	1	1	0	1	0
4'b1111	1	0	1	0	0	1
4'b01xx	0	1	1	0	x	x
4'b01z0	0	1	1	0	x	x

📦 Concatenation & Replication

- **Concatenation:** {A, B, C} joins operands.
- **Replication:** {N{expr}} repeats expr N times.

```
reg A = 1'b1;
reg [1:0] B = 2'b00, C = 2'b10;
reg [2:0] D = 3'b110;
wire [10:0] Y1 = {B, C};                // 4'b0010
wire [10:0] Y2 = {A, B, C, D, 3'b001}; // 11'b10010110001
wire [2:0]  Y3 = {A, B[0], C[1]};      // 3'b101
wire [3:0]  Y4 = {4{A}};               // 4'b1111
wire [7:0]  Y5 = {4{A}, 2{B}};         // 8'b11110000
```

❓ Conditional Operator (?:)

> Syntax: condition ? true_expr : false_expr;

```
assign addr_bus = drive_enable ? addr_out : 36'bz;   // tristate buffer
assign out      = control ? in1 : in0;               // 2-to-1 mux
```

📐 Modeling Styles

➰ Data-Flow Modeling (continuous assignment)

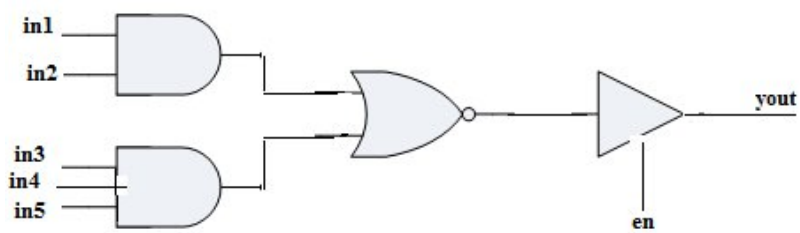
- Uses assign to bind a Boolean expression to a net.
- Example **AND-OR-Invert (AOI)** circuit:

```
module AOI_logic (in1, in2, in3, in4, in5, yout);
    input  in1, in2, in3, in4, in5;
    output yout;
    assign yout = ~((in1 & in2) | (in3 & in4 & in5));
endmodule
```

- Whenever any input changes, the RHS is re-evaluated and the LHS updates automatically.

📷 Visual AOI Example

The diagram shows the two-level AOI structure (AND gates feeding an OR, then inverted).



The picture clarifies how the continuous assignment models the hardware.

🔄 Conditional AOI with Enable

```
module AOI_modified_logic (in1, in2, in3, in4, in5, en, yout);
    input  in1, in2, in3, in4, in5, en;
    output yout;
    assign yout = en ? ~((in1 & in2) | (in3 & in4 & in5)) : 1'bz;
endmodule
```

- When en = 1, the AOI expression drives yout; otherwise yout is high-impedance (z).

⏱️ Propagation Delay on Continuous Assignments

- Delay can be added using # after the net declaration.

```
module AOI_delay_logic (in1, in2, in3, in4, yout);
    input  in1, in2, in3, in4;
    output yout;
    wire #1 y1 = in1 & in2;   // 1-time-unit delay
    wire #1 y2 = in3 & in4;
    wire #1 yout = ~(y1 | y2);
endmodule
```

- Each assignment monitors its RHS and schedules the delayed update to the LHS, mimicking gate-level timing.

📏 Parameterized Modules (Scalable Design)

```
module Mux2_32 (data1, data0, select, muxout);
    parameter word_size = 32;
    input  [word_size-1:0] data1, data0;
    input                  select;
    output [word_size-1:0] muxout;
    assign muxout = select ? data1 : data0;
endmodule
```

- parameter provides a constant that can be changed at instantiation, allowing the same module to support different word widths.